

CCA-F_Study_Guide

Claude Certified Architect - Foundations (CCA-F / CCAR-F)

Complete Study Guide — Explained in Simple Words

Based on the official **Exam Guide Version 1.0, Effective July 2026**, published by Anthropic on the Anthropic Partner Academy.

⚠️ READ THIS FIRST — Important Correction to Your Study Plan

You listed these topics:

- AI Fluency: Framework & Foundations
- Claude 101
- Building with the Claude API
- Claude with Amazon Bedrock
- Claude with Google Cloud Vertex AI
- Introduction to Model Context Protocol
- Claude Code in Action

These are the **Anthropic Academy general course catalog** titles. They are excellent general learning, but they are **not the CCA-F exam blueprint**. Two of them are *explicitly listed as OUT OF SCOPE* in the official exam guide:

Your topic	Exam status
AI Fluency: Framework & Foundations	Not a domain. Background only.
Claude 101	Not a domain. Background only.
Building with the Claude API	Partly in scope (tool_use, tool_choice, stop_reason, Batches API)
Claude with Amazon Bedrock	✗ OUT OF SCOPE — "Specific cloud provider configurations (AWS, GCP, Azure)"
Claude with Google Cloud Vertex AI	✗ OUT OF SCOPE — same reason
Introduction to Model Context Protocol	✓ In scope (Domain 2)
Claude Code in Action	✓ In scope (Domain 3)

What the exam actually tests (the real blueprint):

#	Domain	Weight	~Questions of 60
1	Agentic Architecture & Orchestration	27%	~16
2	Tool Design & MCP Integration	18%	~11
3	Claude Code Configuration & Workflows	20%	~12
4	Prompt Engineering & Structured Output	20%	~12
5	Context Management & Reliability	15%	~9

Adjust your plan: spend ~80% of your time on Domains 1–5 below. Skim Bedrock/Vertex only for general awareness. You will lose real marks if you study Bedrock instead of the Agent SDK.

Exam Format at a Glance

Item	Detail
Exam code	CCAR-F
Questions	60
Format	Multiple-choice AND multiple-response (each item tells you how many to pick)
Structure	4 scenarios , drawn at random from a bank of 6
Time	120 minutes (= 2 minutes per question)
Passing score	720 on a scale of 100–1,000
Fee	\$125 USD
Delivery	Proctored — Pearson VUE (online or test centre)
Validity	12 months
Retakes	14 days → 30 days → 90 days; max 4 attempts per rolling 12 months

The single biggest thing to understand: this is a *judgement* exam, not a *trivia* exam. Most questions give you 4 options where 2–3 are technically real things that would "work." You must pick the one that is **proportionate, addresses the root cause, and uses the right mechanism (deterministic vs probabilistic)**.

The Six Exam Scenarios (memorise these — you get 4 of them)

- Customer Support Resolution Agent** — Agent SDK, MCP tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`), target 80%+ first-contact resolution.
 - Code Generation with Claude Code** — slash commands, CLAUDE.md, plan mode vs direct execution.
 - Multi-Agent Research System** — coordinator + search/analysis/synthesis/report subagents, cited reports.
 - Developer Productivity with Claude** — exploring unfamiliar codebases, built-in tools (Read, Write, Bash, Grep, Glob), MCP servers.
 - Claude Code for CI/CD** — automated code review, test generation, PR feedback, minimising false positives.
 - Structured Data Extraction** — unstructured docs → JSON schema validated output, edge cases.
-

DOMAIN 1 — Agentic Architecture & Orchestration (27%)

This is the biggest domain. If you only have time for one thing, do this one.

1.1 The Agentic Loop — the heart of everything

Simple explanation: An "agent" is just a loop. You send a message to Claude. Claude either answers you, or asks to use a tool. If it asks for a tool, you run the tool, paste the result back into the conversation, and ask Claude again. Repeat until Claude just answers.

How you know which one happened: `stop_reason`.

stop_reason value	Meaning	What your code does
"tool_use"	Claude wants to call a tool	Execute the tool → append result to messages → loop again
"end_turn"	Claude is finished	Stop the loop, show the answer to the user

```
while True:
    response = client.messages.create(model=..., messages=messages, tools=tools)
    messages.append({"role": "assistant", "content": response.content})

    if response.stop_reason == "tool_use":
        results = run_the_requested_tools(response)
        messages.append({"role": "user", "content": results}) # feed results back
        continue
    else: # "end_turn"
        break
```

Key idea: tool results go **back into the conversation history**. That is how Claude "remembers" what it learned and reasons about the next step.

Model-driven vs pre-configured: In a true agentic system, *Claude decides* which tool to call next based on context. This is different from a hard-coded decision tree or a fixed tool sequence. Agentic = adaptive. Workflow = predetermined.

x Anti-patterns the exam will offer you as tempting wrong answers

- Parsing Claude's natural-language text for words like "done" or "finished" to end the loop.
- Using an arbitrary iteration cap (e.g. "stop after 10 loops") as the **primary** stopping mechanism. (*A cap as a safety net is fine; as the main mechanism it is wrong.*)
- Checking whether the response contains text content as a completion signal.

If an option says "check the text for a completion phrase" — it is wrong. The answer is `stop_reason`.

1.2 Multi-Agent Orchestration — Hub and Spoke

Simple explanation: One **coordinator** agent is the boss. It breaks a big task into pieces, hands each piece to a specialist **subagent**, collects the results, and puts them together. Subagents never talk to each other directly — everything goes through the coordinator.

Why hub-and-spoke? Three reasons, and the exam loves these words:

1. **Observability** — you can see everything in one place.
2. **Consistent error handling** — one place decides what to do when something fails.
3. **Controlled information flow** — nothing leaks sideways unexpectedly.

Critical fact: subagents have ISOLATED context. A subagent does **not** automatically inherit the coordinator's conversation history. If the synthesis agent needs the search results, you must **put the search results into its prompt**. This appears repeatedly on the exam.

The coordinator's job:

- Task decomposition (split the work)
- Delegation (choose which subagents to invoke)
- Result aggregation (combine outputs)
- Deciding *dynamically* which subagents to call based on query complexity — **not** always running the whole pipeline every time.

Classic exam trap — narrow decomposition. If the final report is missing whole topic areas, and the coordinator's logs show it only created narrow subtasks, the root cause is **the coordinator's decomposition**, not the subagents. The subagents did their assigned jobs correctly. *Blame the assignment, not the worker.*

Partitioning to avoid duplication: assign each subagent a distinct subtopic or distinct source type (e.g. Agent A = academic papers, Agent B = news) so they don't do the same work twice.

Iterative refinement loop: coordinator reads the synthesis output → spots coverage gaps → re-delegates targeted queries to search/analysis → re-runs synthesis → repeats until coverage is good enough.

1.3 Spawning Subagents — the Task tool

How subagents are created: the **Task tool.**

△ **allowedTools** for the coordinator **MUST include "Task"**, or it cannot spawn subagents at all. This is a favourite exam question. If a scenario says "the coordinator isn't delegating," check **allowedTools** for **Task**.

Parallel subagents: to run subagents in parallel, the coordinator must emit **multiple Task tool calls in a SINGLE response**. If it emits one Task per turn, they run sequentially. This is the whole trick.

AgentDefinition configures each subagent type:

- `description` — when to use this agent
- `system prompt` — its role and behaviour
- tool restrictions — which tools it may use

Passing context properly:

- Put the **complete prior findings directly in the subagent's prompt**.
- Use **structured formats that separate content from metadata** (source URL, document name, page number) so attribution survives the handoff.
- Write coordinator prompts that state **goals and quality criteria**, not step-by-step procedures — this lets subagents adapt.

Fork-based sessions: use `fork_session` to branch from one shared baseline analysis and explore two different approaches in parallel (e.g. "compare refactoring strategy A vs B" from the same codebase understanding).

1.4 Enforcement and Handoffs

The single most important concept in this domain:

Programmatic enforcement (hooks, prerequisite gates) = deterministic guarantee. Prompt instructions (system prompt, few-shot) = probabilistic, non-zero failure rate.

When money, identity, compliance, or safety is involved → **use programmatic enforcement**. Prompt-based instructions will fail some percentage of the time, and "some percentage" is unacceptable for refunds.

Canonical example: Block `process_refund` until `get_customer` has returned a *verified customer ID*. Do this with a prerequisite gate, **not** by telling the model "always verify first."

Structured handoff to humans. When escalating mid-process, the human agent does **not** have access to the conversation transcript. So your handoff summary must contain:

- Customer ID
- Root cause analysis
- Amounts / order numbers / dates
- Recommended action

Multi-concern requests: decompose into distinct items → investigate each **in parallel using shared context** → synthesise **one unified resolution** (not three disconnected answers).

1.5 Agent SDK Hooks

Simple explanation: A hook is a piece of your own code that runs automatically at a specific moment in the agent's lifecycle. It lets you guarantee things instead of hoping the model complies.

Hook use	What it does	Example
PostToolUse	Intercepts the tool <i>result</i> before Claude sees it	Normalise Unix timestamps, ISO-8601 dates, and numeric status codes coming from different MCP tools into one format
Tool call interception	Intercepts the <i>outgoing</i> call before it executes	Block any <code>process_refund</code> over \$500 and redirect to human escalation

Rule of thumb for the exam: "business rule that must *always* hold" → hook. "Style guidance / preference" → prompt.

1.6 Task Decomposition Strategies

Pattern	When to use	Example
Prompt chaining (fixed sequential pipeline)	The steps are known and predictable in advance	Code review: analyse file 1, file 2, file 3 ... then one cross-file integration pass
Dynamic / adaptive decomposition	Open-ended investigation where each step reveals what to do next	"Add comprehensive tests to a legacy codebase": map structure → find high-impact areas → build a prioritised plan that adapts as dependencies appear

Splitting big code reviews: per-file passes for *local* issues + a separate pass for *cross-file* integration. This prevents **attention dilution** and **contradictory findings** (flagging a pattern in one file, approving the identical pattern in another).

1.7 Session State, Resumption, Forking

Command / feature	Purpose
<code>--resume <session-name></code>	Continue a specific named prior conversation
<code>fork_session</code>	Create an independent branch from a shared baseline to explore divergent approaches

When to resume vs when to start fresh:

- **Resume** when prior context is *mostly still valid*. Then explicitly **tell the agent which files changed** so it re-analyses only those.
- **Start a NEW session with a structured summary** when the prior **tool results are stale**. This is more reliable than resuming into out-of-date data. ← exam favourite

DOMAIN 2 — Tool Design & MCP Integration (18%)

2.1 Tool Descriptions Are Everything

The core principle: *Tool descriptions are the PRIMARY mechanism an LLM uses to select a tool.* If two tools have thin, similar descriptions, the model will misroute. The fix is almost always: **write better descriptions first.**

A good tool description contains:

- What the tool does (specific purpose)
- **Input formats** it accepts
- **Example queries**
- **Edge cases**
- **Boundaries** — when to use this tool *versus* the similar-looking one

Classic scenario: `get_customer` ("Retrieves customer information") and `lookup_order` ("Retrieves order details") both accept similar identifiers, and the agent picks the wrong one. **Correct first step = expand both descriptions.** Not few-shot examples (token overhead, doesn't fix root cause). Not a keyword routing layer (over-engineered, bypasses the LLM). Not merging tools (bigger change than a "first step" needs).

Other fixes in the toolkit:

- **Rename** to eliminate overlap: `analyze_content` → `extract_web_results` with a web-specific description.
- **Split generic tools** into purpose-specific ones: `analyze_document` → `extract_data_points`, `summarize_content`, `verify_claim_against_source`.
- **Audit the system prompt** for keyword-sensitive wording that accidentally creates tool associations and overrides good descriptions.

2.2 Structured Error Responses

MCP signals failure with the **isError** flag. But a bare `isError: true` with "Operation failed" is useless — the agent can't decide what to do next.

Return structured error metadata:

```
{
  "isError": true,
  "errorCategory": "transient",    // transient | validation | business | permission
  "isRetryable": true,
  "message": "Order service timed out after 30s. Retry recommended."
}
```

Category	Meaning	Retryable?
transient	Timeout, service temporarily down	✓ Yes
validation	Bad input format	✗ No — fix the input
business	Policy violation (refund window expired)	✗ No — explain to the customer
permission	Not authorised	✗ No

For business errors, include `retryable: false` and a **customer-friendly explanation** so the agent can communicate it kindly rather than retrying pointlessly.

Two distinctions the exam tests:

1. **Retryable vs non-retryable** — structured metadata prevents wasted retries.
2. **Access failure vs valid empty result** — "the search timed out" is completely different from "the search succeeded and found zero matches." Never report them the same way.

Local recovery: subagents should handle transient failures themselves and only propagate to the coordinator what they cannot resolve — including **what was attempted** and **partial results**.

2.3 Tool Distribution & tool_choice

The number-of-tools principle: giving an agent too many tools (e.g. **18 instead of 4-5**) degrades selection reliability because decision complexity increases. Scope each agent's tools to its role.

Why it matters: agents given tools outside their specialisation tend to misuse them (a synthesis agent trying to do web searches).

Scoped cross-role tools: if one agent frequently needs a small capability from another's domain, give it a *narrow* version. Example: give the synthesis agent a `verify_fact` tool for the 85% of simple lookups, and keep routing the 15% complex cases through the coordinator. This is **least privilege** — the exam's preferred answer shape.

Constrained replacements: replace a generic `fetch_url` with `load_document` that validates the URL is actually a document.

tool_choice — memorise this table

Setting	Behaviour
"auto"	Model may call a tool or return plain text. Default.
"any"	Model must call a tool, but chooses which one
{"type": "tool", "name": " <code>extract_metadata</code> "}	Model must call that exact tool

- Use `"any"` when you need guaranteed structured output but the document type is unknown (multiple schemas available).
- Use forced selection to guarantee a specific tool runs **first** (e.g. `extract_metadata` before enrichment), then handle later steps in follow-up turns.

2.4 MCP Server Configuration

Scope	File	Use for
Project	<code>.mcp.json</code> (in repo)	Shared team tooling — committed to version control
User	<code>~/.claude.json</code>	Personal / experimental servers — not shared

Credentials: use **environment variable expansion** in `.mcp.json` so secrets are never committed:

```

{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" }
    }
  }
}

```

Key facts:

- Tools from **all** configured MCP servers are discovered at connection time and are available **simultaneously**.
- MCP resources** expose *content catalogs* (issue summaries, documentation hierarchies, database schemas) so the agent can *see what data exists* without burning turns on exploratory tool calls.
 - Tools = actions. Resources = content/catalogs.**
- If the agent keeps preferring built-in **Grep** over your more capable MCP tool → **improve the MCP tool's description** to explain its capabilities and outputs in detail.
- Prefer an **existing community MCP server** for standard integrations (Jira, GitHub). Build custom servers only for team-specific workflows.

2.5 Built-in Tools

Tool	Use it for	Memory hook
Grep	Searching file contents — function names, error strings, import statements	"Grep = what's <i>inside</i> "
Glob	Finding files by path/name pattern — <code>**/*.test.tsx</code>	"Glob = what it's <i>called</i> "
Read	Load a full file's contents	
Write	Write a whole file	
Edit	Targeted change using unique text matching	Fails if the anchor text isn't unique
Bash	Run shell commands	

Edit failure fallback: when **Edit** fails because the anchor text appears more than once → use **Read + Write**.

Incremental exploration pattern (exam favourite): Do **not** read every file upfront. Start with **Grep** to find entry points → **Read** to follow imports and trace flows → expand only as needed. Reading everything upfront exhausts context.

Tracing usage across wrapper modules: first identify all *exported names*, then Grep for each name across the codebase.

DOMAIN 3 — Claude Code Configuration & Workflows (20%)

3.1 The CLAUDE.md Hierarchy

Level	Path	Shared with team?
User	<code>~/.claude/CLAUDE.md</code>	✗ No — personal only, not in version control
Project	<code>.claude/CLAUDE.md</code> or root <code>CLAUDE.md</code>	✓ Yes — committed to the repo
Directory	<code>CLAUDE.md</code> inside a subdirectory	✓ Yes — applies to that directory

Classic diagnostic question: "A new team member isn't getting our coding standards." → The standards were written in **user-level** `~/.claude/CLAUDE.md` instead of project-level. Move them to the project level.

Keeping CLAUDE.md modular:

- **@import syntax** — reference external files so each package's CLAUDE.md only pulls in the standards relevant to it.
- **.claude/rules/** — split a giant CLAUDE.md into topic files: `testing.md`, `api-conventions.md`, `deployment.md`.

/memory command — shows which memory files are currently loaded. Use it to diagnose "Claude behaves differently in different sessions."

3.2 Slash Commands and Skills

Thing	Project scope (shared)	User scope (personal)
Slash commands	<code>.claude/commands/</code>	<code>~/.claude/commands/</code>
Skills	<code>.claude/skills/</code>	<code>~/.claude/skills/</code>

Skills live in `.claude/skills/<name>/SKILL.md` and support **frontmatter**:

```

---
name: analyze-codebase
description: Deep structural analysis of a package
context: fork          # run in an isolated sub-agent context
allowed-tools: [Read, Grep, Glob] # restrict tool access during execution
argument-hint: "<package-name>"  # prompt the developer for required args
---
```

Frontmatter option	What it does
<code>context: fork</code>	Runs the skill in an isolated sub-agent context , so verbose or exploratory output does not pollute the main conversation
<code>allowed-tools</code>	Restricts which tools the skill can use (e.g. limit to file writes to prevent destructive actions)
<code>argument-hint</code>	Prompts the developer for required parameters if they invoke the skill bare

When to use `context: fork`: verbose output (codebase analysis) or exploratory context (brainstorming alternatives) that you don't want in the main session.

Personal customisation without affecting teammates: create a variant with a different name in `~/.claude/skills/`.

Skills vs CLAUDE.md — the decision rule

- **CLAUDE.md** = *always loaded*, universal standards.
- **Skills** = *on-demand invocation*, task-specific workflows.

3.3 Path-Specific Rules

`.claude/rules/*.md` files can carry YAML frontmatter with a `paths` field containing **glob patterns**:

```
---
paths: ["**/*.test.tsx", "**/*.test.ts"]
---
All tests use React Testing Library. No enzyme. Use `describe`/'it' naming...
```

Why this beats directory-level CLAUDE.md: rules load only when editing matching files (saving tokens and irrelevant context), and glob patterns work **regardless of directory location**. Test files scattered across the whole codebase next to their source files can't be covered by directory-bound CLAUDE.md — but `**/*.test.tsx` covers them all.

Decision rule: conventions tied to a **file type spread across directories** → path-specific rules. Conventions tied to **one directory** → directory CLAUDE.md.

3.4 Plan Mode vs Direct Execution

Use plan mode when...	Use direct execution when...
Large-scale changes (45+ files, monolith → microservices)	Simple, well-scoped change
Multiple valid approaches exist	One obvious approach
Architectural decisions required	No architecture implications
Multi-file modifications	Single-file fix with a clear stack trace
Library migrations, choosing between integration approaches	Adding one validation conditional to one function

Plan mode lets you **explore safely and design before committing to changes**, preventing costly rework.

Combining them: plan mode for the *investigation*, then direct execution for the *implementation* of the agreed plan.

Explore subagent: use it during verbose discovery phases. It isolates the noisy exploration output and returns only a **summary**, preserving the main conversation's context window.

Exam trap: "Start in direct execution and switch to plan mode if it gets complicated" is **wrong** when the requirements already state the task is complex. The complexity is known upfront.

3.5 Iterative Refinement Techniques

Technique	When it's the right answer
Concrete input/output examples (2-3)	Prose descriptions are being interpreted inconsistently. Most effective way to communicate an expected transformation.
Test-driven iteration	Write the test suite first (expected behaviour, edge cases, performance), then iterate by sharing the test failures
Interview pattern	Unfamiliar domain — have Claude ask you questions to surface considerations you hadn't thought of (cache invalidation, failure modes) <i>before</i> implementing
Single detailed message with all issues	The problems interact with each other
Sequential one-at-a-time fixes	The problems are independent

Edge-case bugs (e.g. nulls in a migration script) → give a **specific test case with example input and expected output**.

3.6 Claude Code in CI/CD

Flag	Purpose
<code>-p / --print</code>	Non-interactive mode. Processes the prompt, prints to stdout, exits. Without it, CI jobs hang waiting for input.
<code>--output-format json</code>	Machine-parseable output
<code>--json-schema</code>	Enforce a specific structured output shape

```
claude -p "Review this diff for security issues" \
  --output-format json \
  --json-schema ./review-schema.json
```

There is **no** `CLAUDE_HEADLESS` env var and **no** `--batch` flag. These are distractor options on the exam.

CI best practices from the blueprint:

- **CLAUDE.md carries the project context** into CI-invoked runs: testing standards, fixture conventions, review criteria, what counts as a valuable test. This is how you reduce low-value test output.
- **Avoid duplicate PR comments:** when re-running a review after new commits, include the **prior review findings** in context and instruct Claude to report only **new or still-unaddressed** issues.
- **Avoid duplicate tests:** provide the **existing test files** in context so generation doesn't re-cover what's already tested.
- **Session context isolation:** the same session that *generated* code is worse at *reviewing* it. Use an **independent instance** for review.

DOMAIN 4 — Prompt Engineering & Structured Output (20%)

4.1 Explicit Criteria Beat Vague Instructions

The principle: vague quality instructions do not reduce false positives. Specific categorical criteria do.

✗ Doesn't work	✓ Works
"Be conservative"	"Report bugs and security issues; skip minor style and locally-consistent patterns"
"Only report high-confidence findings"	"Flag a comment only when the claimed behaviour contradicts the actual code behaviour"
"Check that comments are accurate"	Concrete code examples for each severity level

False positives destroy trust. One high-false-positive category undermines developer confidence in the *accurate* categories too. Legitimate tactic: **temporarily disable the noisy category** to restore trust while you improve its prompt.

Consistent severity classification requires explicit severity criteria **with concrete code examples for each level.**

4.2 Few-Shot Prompting

When few-shot is the correct answer: when detailed *instructions alone* are producing inconsistent results, and you need **format consistency** or **judgement on ambiguous cases**.

Use **2-4 targeted examples** that **show the reasoning** for why one action was chosen over a plausible alternative. That reasoning is what lets the model **generalise to novel patterns** rather than only matching your listed cases.

Few-shot is the go-to for:

- Consistently formatted, actionable output (location / issue / severity / suggested fix)
- Ambiguous tool selection cases
- Distinguishing acceptable code patterns from genuine issues (false-positive reduction *with* generalisation)
- Varied document structures in extraction (inline citations vs bibliographies; methodology sections vs embedded details)
- Reducing hallucination / empty extraction of required fields

Few-shot vs better tool descriptions: if the problem is *tool selection with thin descriptions*, fix the **descriptions** first. If the problem is *ambiguous judgement or inconsistent formatting*, use **few-shot**.

4.3 Structured Output via tool_use + JSON Schema

The most reliable way to get guaranteed schema-compliant output is tool use with a JSON schema. It eliminates JSON syntax errors entirely.

```

tools = [{
  "name": "extract_invoice",
  "description": "Record extracted invoice fields",
  "input_schema": {
    "type": "object",
    "properties": {
      "invoice_number": {"type": "string"},
      "vendor_name": {"type": ["string", "null"]},          # nullable – may be absent
      "category": {"enum": ["goods", "services", "other", "unclear"]},
      "category_detail": {"type": ["string", "null"]},     # "other" + detail pattern
      "stated_total": {"type": "number"},
      "calculated_total": {"type": "number"}              # for self-validation
    },
    "required": ["invoice_number", "stated_total"]
  }
}]

```

⚠ The most important nuance in this domain

Strict JSON schemas eliminate SYNTAX errors. They do NOT prevent SEMANTIC errors. Line items that don't sum to the stated total, a date placed in the wrong field, a value that's plausible but wrong — the schema will happily accept all of these. Semantic correctness needs **validation logic**, not a schema.

Schema design rules

Rule	Why
Make fields optional / nullable when the source may not contain them	Prevents the model fabricating values to satisfy a required field
Add an "unclear" enum value	Gives the model an honest option for ambiguous cases
Use "other" + a detail string	Extensible categorisation without breaking the enum
Put format normalisation rules in the prompt	Schema controls shape; the prompt handles messy source formatting

4.4 Validation, Retry, and Feedback Loops

Retry-with-error-feedback: on failure, send a follow-up containing **(1) the original document, (2) the failed extraction, (3) the specific validation errors**. This lets the model self-correct.

When retry works vs when it's pointless

Retry will help	Retry will NOT help
Format mismatches	The information simply isn't in the source document
Structural output errors	The data lives in an external document you didn't provide
Wrong field placement	

Recognising "no amount of retrying will conjure data that isn't there" is a tested judgement.

Self-correction schema patterns:

- Extract **calculated_total** **alongside** **stated_total** → flag discrepancies automatically.
- Add a **conflict_detected** boolean for inconsistent source data.
- Add a **detected_pattern** field to each finding → when developers dismiss findings, you can systematically analyse *which code constructs* trigger false positives.

4.5 Message Batches API

Property	Value
Cost saving	50%
Processing window	up to 24 hours
Latency SLA	None guaranteed
Correlation	<code>custom_id</code> links requests to responses
Multi-turn tool calling within one request	✗ Not supported

Use batch for: overnight reports, weekly audits, nightly test generation — anything latency-tolerant and non-blocking. **Never use batch for:** blocking workflows where a human is waiting — pre-merge checks, interactive review.

Standard exam question: "switch both the blocking pre-merge check and the overnight tech-debt report to batch." Answer: **batch the overnight report only**, keep the pre-merge check synchronous.

SLA maths: if batches take up to 24 hours and you must guarantee a 30-hour SLA, submit every ≤ 6 hours (the guide's worked example uses **4-hour submission windows** for a 30-hour SLA — build in margin).

Failure handling: resubmit **only the failed documents**, identified by `custom_id`, with fixes applied (e.g. chunking documents that blew the context limit).

Cost tip: refine your prompt on a **small sample** before batching thousands of documents — maximises first-pass success and avoids expensive resubmission cycles.

4.6 Multi-Instance and Multi-Pass Review

Self-review limitation (know this cold): a model that generated the code **retains its reasoning context** and is therefore *less likely to question its own decisions* in the same session.

Therefore: an **independent review instance with no prior reasoning context** catches subtle issues better than:

- telling the same session "now review your work critically", or
- turning on extended thinking.

Multi-pass review for large PRs: per-file passes for local issues + a separate cross-file integration pass. This avoids **attention dilution** and **contradictory findings**.

Trap: "use a bigger context window model" does **not** fix attention dilution. Trap: "run 3 passes and only flag issues found in 2 of 3" **suppresses real bugs** that are caught intermittently.

Confidence in review: have the model self-report confidence **alongside each finding** to enable calibrated routing. (Note the contrast with Domain 5: self-reported confidence is fine for *routing attention*, but unreliable as an *escalation trigger*.)

DOMAIN 5 — Context Management & Reliability (15%)

5.1 Preserving Critical Information

Progressive summarisation is dangerous. Each round of summarising erodes the precise things that matter most: **numbers, percentages, dates, order numbers, and customer-stated expectations** get condensed into vague prose ("the customer was unhappy about the amount").

The fix — a persistent "case facts" block. Extract transactional facts into a structured block that is included in **every** prompt, **outside** the summarised history:

```
CASE FACTS (do not summarise):
customer_id: C-88213
order: #12345, placed 2026-03-04, $249.99
issue: damaged on arrival, photo provided
customer expectation: full refund, not store credit
```

"Lost in the middle": models reliably process information at the **beginning** and **end** of long inputs, and may omit findings from the **middle**. **Mitigations:** put the **key findings summary at the beginning** of aggregated input, and organise details with **explicit section headers**.

Verbose tool outputs: an order lookup returning 40+ fields when only 5 are relevant burns context disproportionately. **Trim tool outputs to the relevant fields before they accumulate.**

Upstream/downstream: when a downstream agent has a tight context budget, modify the **upstream** agent to return **structured data (key facts, citations, relevance scores)** instead of verbose content and reasoning chains.

Also: always pass the **complete conversation history** in subsequent API requests to maintain coherence.

5.2 Escalation and Ambiguity Resolution

The three legitimate escalation triggers

1. The **customer explicitly asks for a human**.
2. A **policy exception or policy gap** — the policy is silent or ambiguous on this request (e.g. competitor price matching when the policy only covers own-site adjustments). *Note: "the case is complex" is not by itself a trigger.*

3. The agent **cannot make meaningful progress**.

Unreliable escalation triggers (all wrong answers)

- **✗ Sentiment analysis / frustration detection** — sentiment does not correlate with case complexity.
- **✗ Model self-reported confidence scores** — poorly calibrated; the agent is *already* wrongly confident on the hard cases.
- **✗ A separately trained classifier** — over-engineered before prompt optimisation has been tried.

The proportionate first fix for bad escalation calibration: add **explicit escalation criteria with few-shot examples** showing escalate-vs-resolve, to the system prompt.

Nuance on explicit requests: if the customer *demand*s a human → escalate **immediately**, don't investigate first. If the customer is merely frustrated but the issue is within your capability → **acknowledge the frustration and offer to resolve**, escalating only if they reiterate the preference.

Multiple matches: if a tool returns several matching customers, **ask for an additional identifier**. Never pick one by heuristic.

5.3 Error Propagation Across Multi-Agent Systems

Return structured error context to the coordinator:

- Failure type
- What was attempted (the query)
- Any partial results
- Potential alternative approaches

This is what lets the coordinator decide intelligently: retry with a modified query, try an alternative, or proceed with partial results and annotate the gap.

The four anti-patterns

Anti-pattern	Why it's wrong
Generic status: "search unavailable"	Hides context the coordinator needs
Returning empty results marked successful	Silently suppresses the error; risks silently incomplete output
Terminating the whole workflow on one failure	Recovery strategies could have succeeded
Confusing access failure with valid empty result	Completely different meanings; different responses

Local recovery first: subagents handle transient failures themselves; only propagate what they can't resolve, *with* what was attempted and partial results.

Coverage annotations: structure synthesis output so it states which findings are **well-supported** and which topic areas have **gaps due to unavailable sources**.

5.4 Context in Large Codebase Exploration

Context degradation symptom to recognise: in long sessions the model starts giving **inconsistent answers** and referring to "**typical patterns**" rather than the **specific classes it discovered earlier**. That phrase in a question stem is your signal.

The toolkit

Technique	What it solves
Scratchpad files	Persist key findings across context boundaries; the agent re-reads them for later questions
Subagent delegation	Isolate verbose exploration output; the main agent keeps only high-level coordination
Phase summarisation	Summarise findings from phase 1, inject the summary into phase 2's initial context
/compact	Reduce context usage mid-session when it's filled with verbose discovery output
Structured state exports + manifest	Crash recovery — each agent exports state to a known location; the coordinator loads the manifest on resume and injects it into agent prompts

5.5 Human Review Workflows & Confidence Calibration

The headline risk: an aggregate accuracy figure like **97% overall** can mask **terrible performance on one document type or one field**. Before you reduce human review, **analyse accuracy by document type and by field segment**.

Stratified random sampling of *high-confidence* extractions gives you ongoing error-rate measurement and detects **novel error patterns** that your existing checks don't catch.

Field-level confidence scores, calibrated against a **labelled validation set**, let you set review thresholds and route reviewer attention where it pays off.

Route to human review: low model confidence, **or** ambiguous/contradictory source documents. Reviewer capacity is limited — prioritise.

5.6 Provenance and Uncertainty in Synthesis

How attribution gets lost: during summarisation steps, findings are compressed and the **claim → source mapping** is dropped.

The fix: require subagents to output **structured claim-source mappings**:

```
{
  "claim": "Adoption grew 34% year over year",
  "evidence_excerpt": "...adoption rose 34% between 2024 and 2025...",
  "source": "https://example.org/report-2025",
  "document_name": "Industry Report 2025",
  "publication_date": "2025-11-02"
}
```

...and require downstream agents to **preserve and merge** these mappings through synthesis.

Conflicting statistics from two credible sources: annotate the conflict with source attribution for both values. Do **not** arbitrarily pick one. Let the coordinator decide how to reconcile before synthesis. Structure the report with **explicit sections separating well-established findings from contested ones**, preserving each source's original characterisation and methodological context.

Temporal data: require **publication or data-collection dates** in structured outputs, so a 2023 figure and a 2025 figure aren't misread as a contradiction.

Rendering: render each content type appropriately in synthesis — **financial data as tables, news as prose, technical findings as structured lists** — rather than forcing everything into one uniform format.

Background Topics (Your Original List) — Quick Coverage

These aren't tested as domains, but here's the essential background so nothing feels unfamiliar.

AI Fluency: Framework & Foundations

Anthropic's **4D framework** for working effectively with AI: **Delegation** (decide what to hand to AI vs do yourself), **Description** (communicate the task clearly), **Discernment** (critically evaluate the output), **Diligence** (use it responsibly and transparently). Useful mental model; not directly examined.

Claude 101

Claude is a family of large language models. Practical basics: the **system prompt** sets role and behaviour; **messages** alternate user/assistant; **max_tokens** caps the response length; **context window** is the total budget for input + output. Model tiers trade off capability, speed, and cost. *Out of scope: internal architecture, training, Constitutional AI, RLHF.*

Building with the Claude API — *only these parts are in scope*

- `tool_use` blocks and JSON `input_schema`
- `tool_choice`: `auto` / `any` / forced
- `stop_reason`: `tool_use` / `end_turn`
- `max_tokens`, system prompts
- Message Batches API
- **✗ Explicitly out of scope:** authentication, billing, rate limits, pricing calculations, streaming/SSE, token counting, prompt caching internals.

Claude with Amazon Bedrock / Google Cloud Vertex AI

✗ **Explicitly out of scope: "Specific cloud provider configurations (AWS, GCP, Azure)."** Worth one paragraph of awareness only: Claude is available through AWS Bedrock and GCP Vertex AI as alternative deployment surfaces to the first-party Anthropic API, letting enterprises keep traffic inside their existing cloud contract and data boundary. The message format is broadly the same; the auth and endpoint differ. **Do not spend study time here.**

Introduction to MCP — see Domain 2

MCP is an open protocol that standardises how AI applications connect to external systems. A **client** (Claude Code, an agent) connects to **servers** that expose **tools** (actions the model can invoke), **resources** (content the model can read), and **prompts** (reusable templates). It replaces N×M bespoke integrations with one standard.

Claude Code in Action — see Domain 3

Anthropic's agentic coding tool for the terminal, desktop, and IDEs. Everything examinable is in Domain 3 above.

Study Plan (2 Weeks)

Days	Focus
1-3	Domain 1 (27%). Actually build an agentic loop with <code>stop_reason</code> handling. Spawn a subagent via <code>Task</code> .
4-5	Domain 2 (18%). Write 4 MCP tools with genuinely differentiating descriptions. Add structured error categories.
6-8	Domain 3 (20%). Configure a real repo: CLAUDE.md hierarchy, <code>.claude/rules/</code> with globs, a skill with <code>context: fork</code> , an <code>.mcp.json</code> . Run <code>claude -p</code> in a CI job.
9-11	Domain 4 (20%). Build an extraction pipeline with <code>tool_use</code> + JSON schema, nullable fields, and a validation-retry loop.
12-13	Domain 5 (15%). Practise context trimming, scratchpads, escalation criteria.
14	Full 60-question mock under timed conditions. Review every wrong answer's <i>justification</i> , not just the letter.

How to Answer Exam Questions — Decision Heuristics

1. **Root cause, not symptom.** If the logs point at one component, that's the answer — don't blame downstream agents doing their job correctly.
2. **Proportionate response.** Prefer the smallest effective fix. "Train a classifier" / "build a routing layer" / "upgrade the model" are usually over-engineered distractors.
3. **Deterministic vs probabilistic.** Money, identity, compliance → hooks/gates. Style, judgement → prompts.
4. **Descriptions before examples.** Tool-selection problems → fix descriptions. Format/judgement problems → few-shot.
5. **Independent instances beat self-review.**
6. **Least privilege.** Scope tools narrowly; add a *scoped* cross-role tool rather than full access.

7. **Never silently swallow an error.** Structured error context is nearly always the right answer.
8. **Explicit context passing.** Subagents inherit nothing.
9. **Watch for non-existent features.** `--batch`, `CLAUDE_HEADLESS`, `.claude/config.json` commands array — all fake.
10. **Read the stem for the constraint.** "first step", "most maintainable", "most effective", "with least effort" changes which correct-looking option wins.

*Prepared as independent study material. The authoritative source is the official Exam Guide on the Anthropic Partner Academy:
<https://anthropic-partners.skilljar.com/claude-certified-architect-foundations-certification>*